

ML Accelerator Tabular Data1

martes, 8 de junio de 2021 11:03

Learning Outcomes

- Fundamental understanding of data preprocessing, commonly used machine learning (ML) algorithms, and model evaluation *data preparation* *performance*
- Gain familiarity with standard ML tools and data preprocessing libraries, including methods for detecting and remedying overfitting *Python*
- Be comfortable talking with scientist partners *SageMaker notebooks*
- Learn about valuable and useful ML resources at Amazon *end to end ML lifecycle*

Course Overview

<u>Lecture 1</u>	<u>Lecture 2</u>	<u>Lecture 3</u>
- Introduction to ML / <i>or</i> - Model Evaluation - Train-Validation-Test - Overfitting <i>EVA</i> - Exploratory Data Analysis - K Nearest Neighbors (KNN) <i>(D)</i>	- Feature Engineering - Tree-based Models - Decision Tree - Random Forest <i>(ensemble)</i> - Hyperparameter Tuning - AWS AI/ML Services <i>SM</i>	- Optimization - Regression Models - Regularization - Boosting <i>(ensemble)</i> - Neural Networks <i>(3)</i> - AutoML <i>(2)</i>

Machine Learning Applications

Business/ML Problem	Description	Example
Ranking	Helping users find the most relevant thing	
Recommendation	Giving users the thing they may be most interested in	
Classification	Figuring out what kind of thing something is	
Regression	Predicting a numerical value of a thing	
Clustering	Putting similar things together	
Anomaly Detection	Finding uncommon things	

Machine Learning Applications

Business/ML Problem	Description	Example
Ranking	Helping users find the most relevant thing	
Recommendation	Giving users the thing they may be most interested in	
Classification	Figuring out what kind of thing something is	
Regression	Predicting a numerical value of a thing	
Clustering	Putting similar things together	
Anomaly Detection	Finding uncommon things	

Machine Learning Applications

Business/ML Problem	Description	Example
Ranking	Helping users find the most relevant thing	
Recommendation	Giving users the thing they may be most interested in	
Classification	Figuring out what kind of thing something is	
Regression	Predicting a numerical value of a thing	
Clustering	Putting similar things together	
Anomaly Detection	Finding uncommon things	

Machine Learning Applications

Business/ML Problem	Description	Example
Ranking	Helping users find the most relevant thing	
Recommendation	Giving users the thing they may be most interested in	
Classification	Figuring out what kind of thing something is	
Regression	Predicting a numerical value of a thing	
Clustering	Putting similar things together	
Anomaly Detection	Finding uncommon things	

Machine Learning Applications

Business/ML Problem	Description	Example
Ranking	Helping users find the most relevant thing	
Recommendation	Giving users the thing they may be most interested in	
Classification	Figuring out what kind of thing something is	
Regression	Predicting a numerical value of a thing	
Clustering	Putting similar things together	
Anomaly Detection	Finding uncommon things	

Machine Learning Applications

Business/ML Problem	Description	Example
---------------------	-------------	---------

Ranking	Helping users find the most relevant thing
Recommendation	Giving users the thing they may be most interested in
Classification	Figuring out what kind of thing something is
Regression	Predicting a numerical value of a thing
Clustering	Putting similar things together
Anomaly Detection	Finding uncommon things



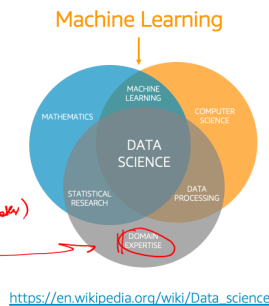
What is Machine Learning?

What is Data Science?

Wikipedia describes **Data Science** as:

multiple skills
 "a multi-disciplinary field that uses scientific methods, processes, algorithms and systems to extract knowledge and insights from structured and unstructured data."

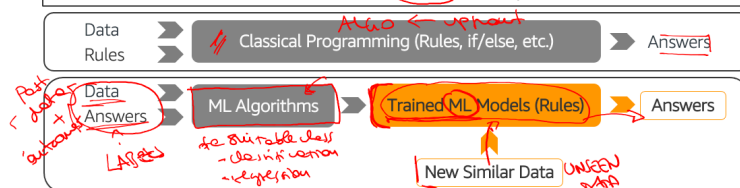
1. *Domain Teams* - scientists (ML/ML), engineers, SRE
2. *ML = learning from data*



What is Machine Learning?

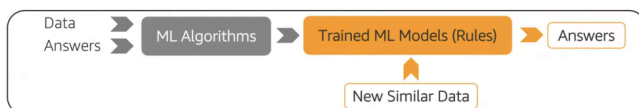
"Programming computers to learn from experience should eventually eliminate the need for much of this detailed programming effort"

Arthur Samuel (1959) - Computer Scientist

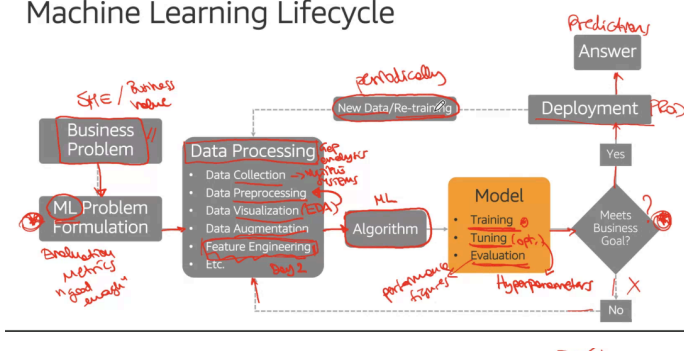


Why ML? Why now?

- Data** *Big Data*
 - larger amounts of data, easy to produce, *cheaper* collect and store
 - Compute** *GPU*
 - powerful processing units, hardware acceleration, parallelization
 - Algorithms** *High level*
 - ML frameworks, libraries, improved and more efficient techniques
- ML approaches everyday things*
Equation relations



Machine Learning Lifecycle



Data Augmentation --> cuando no se tienen demasiados casos reales, hacer oversampling o Synthetic Data.

Data Enrichment --> Add new possible useful sources of data

Some Important ML Terms

Regression Example: Predicting house prices based on # of rooms, # of bathrooms, # of bedrooms, # of floors, volume

ML	Statistics/Math/other	Simply Put
<u>Label/Target/y</u>	Dependent/Response/Output Variable	The thing you're trying to predict
<u>Feature/x</u>	Independent/Explanatory/Input Variable	Data that help you make predictions
<u>Feature Engineering</u>	<u>Transformation</u>	Reshaping data to get more value
1d, 2d, ... nd	<u>Dimensionality</u>	Number of features
<u>Model Parameters</u>	<u>Weights</u>	A set of numbers embedded in a model that can predict the labels
<u>Model Training</u>	<u>Optimization</u>	Finding the 'best' set of model parameters

Ejemplo de transformation -> Desde el año de construcción sacar los años que tiene la casa, que es más útil en un modelo de ML como valor numérico que una fecha

Supervised & Unsupervised Learning

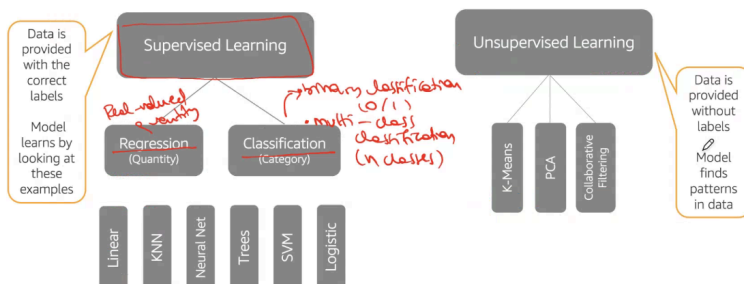
Supervised vs. Unsupervised Learning

Business/ML Problem	Description
Ranking	Helping users find the most relevant thing
Recommendation	Giving users the thing they may be most interested in
<u>Classification</u>	Figuring out what kind of thing something is
<u>Regression</u>	Predicting a numerical value of a thing
<u>Clustering</u>	Putting similar things together
Anomaly Detection	Finding uncommon things

Supervised Learning: Data is provided with the correct labels

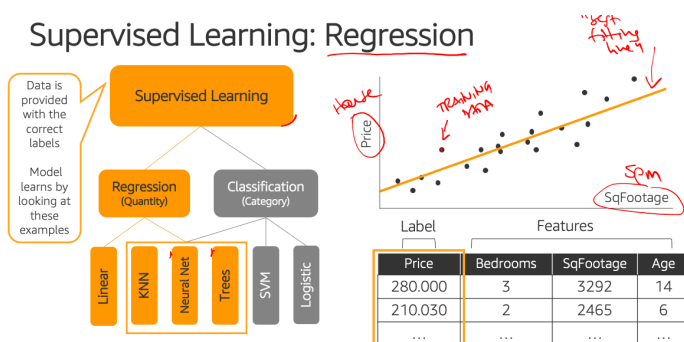
Unsupervised Learning: Data is provided without labels

Supervised vs. Unsupervised Learning



En la regresión tenemos valores numéricos, en la clasificación tenemos variables categóricas binarias o múltiples.

Supervised Learning: Regression



Con algoritmos KNN, NN and Trees pueden hacer tanto regresión como clasificación

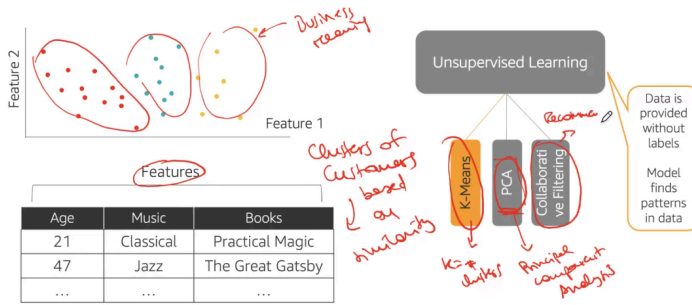
Supervised Learning: Classification





1	5	10<	750
0	0	>9	150
...	...	...	...

Unsupervised Learning: Clustering

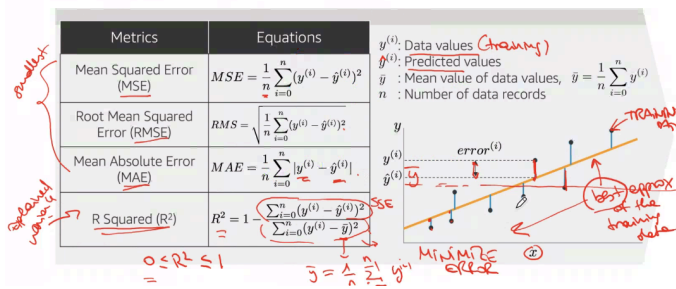


Model Evaluation

Quantitatively measure model's performance

SUPERVISED LEARNING

Regression Metrics



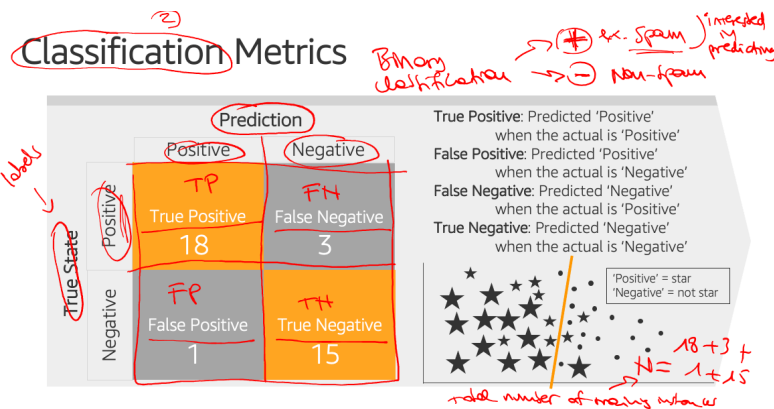
R cuadrado

Sum of square errors

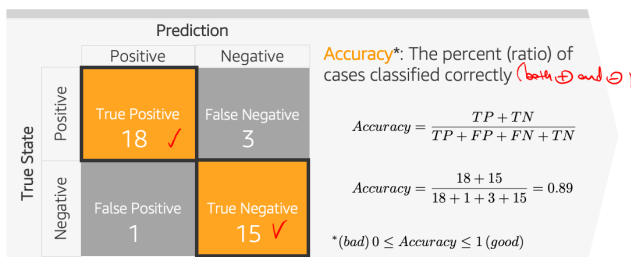
Sum of square errors against the average

Si da 1, realmente significa que no es bueno, ya que es muy probable que esté sobre entrenado, no es normal que nos de para todos valores predichos el mismo error frente a los valores reales que la media de todos los valores predichos frente a los valores reales

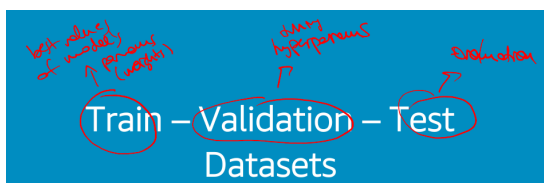
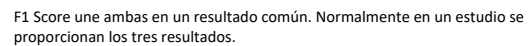
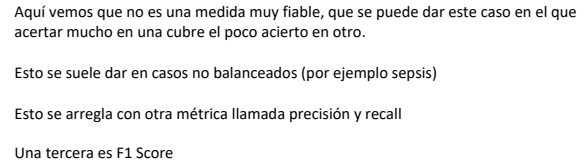
Classification Metrics



Classification Metrics: Accuracy



lot more
true negatives than
↑ true
positives



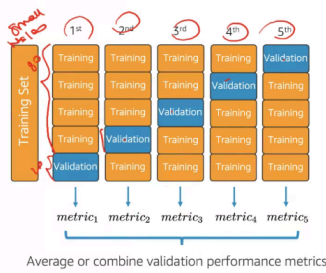
• How to make sure our model generalizes well?

- Shuffle the dataset before the split!

en
ho.

K-fold Cross Validation ($K=5$)

Embedded in most libraries



Use K different holdout samples to validate the model, each time training with the remainder samples:

- Split the training dataset into K independent folds.
- Repeat the following K times:
 - Set aside K^{th} fold of the data for **validation**.
 - Train the model on the other folds, the **training set**.
 - Test the model on the **validation set**.
- Average or combine the model performance metrics.

En los algoritmos puede que esta K esté como parámetro, que es el número de folders. De lo que se trata es de validar cada folder en el que se ha dividido el training contra el resto de folders, por eso sale un matriz de $K \times K$. El número de folders está muy relacionado con el tamaño del dataset de entrenamiento.

Underfitting & Overfitting

Model Evaluation: Underfitting

1. Underfitting
2. Overfitting
3. Good Fit

Underfitting: Model is not good enough to describe the relationship between the input data (x_1, x_2) and output y : {Class 1, Class 2}.



- Model is **too simple** to capture important patterns of training data.
- Model will **perform poorly on training and validation** (and/or test).
- Corresponding to **high bias**, results show systematic lack of fit in certain regions.

Model Evaluation: Overfitting

Overfitting: Model memorizes or imitates **training data**, and fails to generalize well on new "unseen" data (test data).



- Model is **too complex**, picks up the noise instead of the patterns of training data.
- Model will **perform well on training, but poorly on validation** (and/or test).
- Corresponding to **high variance**, small changes in the data lead to big result changes.

Model Evaluation: Good Fit

Appropriate fitting: Model captures the general relationship between the input data (x_1, x_2) and output y : {Class 1, Class 2}.



- Model not too simple, not too complex.
- Model **picks up the underlying relationship** rather than the noise in the training.
- Model will **perform good enough on training and validation** (and/or test).
- Bias-variance trade-off.

Aquí sabemos que hay un par de puntos que son "Ruido" porque se salen del modelo pero contamos con ello. Un buen fit es un buen balance entre variance y Bias

Exploratory Data Analysis (EDA)

Data processing
→ model

Exploratory Data Analysis

Exploratory Data Analysis (EDA) is an approach to analyze a dataset and capture its main characteristics of it.

and capture main characteristics of it.

- Collect or aggregate data
- Perform initial investigations to discover patterns, spot anomalies, test hypothesis and check assumptions
 - Summary statistics
 - Graphical/visual representations (histograms, plots)
- Process data to produce meaningful information

Descriptive Statistics

Overall statistics - `df.head()`, `df.shape`, `df.info()`

- Number of instances (i.e. number of rows)
- Number of features (i.e. number of columns)

* Univariate statistics (single feature)

- Statistics for numerical features (mean, variance, histogram) - `df.describe()`, `hist(df[feature])`
- Statistics for categorical features (histograms, mode, most/least frequent values, percentage, number of unique values)

▪ Histogram of values - `df[feature].value_counts()` or `seaborn's distplot()`

• Target statistics

▪ Class distribution - `df[target].value_counts()` or `np.bincount(y)`

* Multivariate statistics (more than one feature)

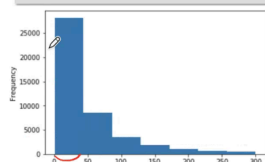
- Correlations - `df.plot.scatter(feature1, feature2)`, `df[[feature1, feature2]].corr()`

1 fila por instancia y 1 columna por feature

Univariate Statistics: Histograms

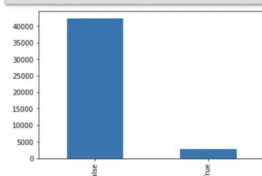
Numerical features:

```
import matplotlib.pyplot as plt
df[num_feature].plot.hist(bins = 7)
plt.show()
```



Categorical features:

```
import matplotlib.pyplot as plt
df[cat_feature].value_counts().plot.bar()
plt.show()
```

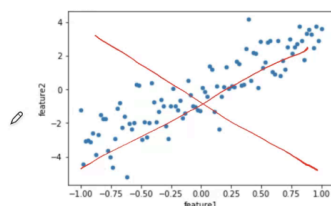
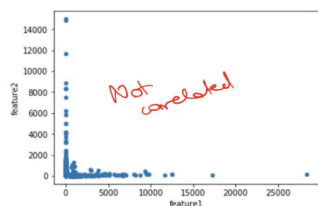


Correlations: Scatterplot

Correlations: How strongly pairs of features are related.

```
df.plot.scatter(feature1, feature2)
plt.show()
```

Scatterplot matrices **visualize** feature-target and feature-feature pairwise relationships.



Correlación positiva o negativa dependiendo de la inclinación de la recta

Correlations: Correlation Matrix

Correlations: How strongly pairs of features are related.

```
cols = [feature1, feature2]
df[cols].corr()
```

Correlation matrices **measure** the linear dependence between features; easier to read; can use heat-maps.

	feature1	feature2
feature1	1	0.0128493
feature2	0.0128493	1

	feature1	feature2
feature1	1	0.882106
feature2	0.882106	1

Correlation **values** are between -1 and 1: -1 means perfect negative correlation, 1 means perfect positive correlation, and 0 means there is no relationship between the two variables.

Correlations

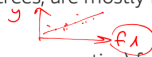
Highly correlated (positive or negative) features might degrade

Si tenemos muchos features correlacionados, lo mejor sería eliminar uno de ellos ya que nos puede afectar al rendimiento del modelo metiendo ruido sin aportar información adicional

performance of some ML models, such as linear and logistic regression models.

- Select one of the correlated features and discard the other(s).
- Other ML models, like decision trees, are mostly immune to this problem.

While, highly target-correlated (positive or negative) features might improve the performance of linear and logistic regression models.



Sin embargo si estas correlación es con la variable objetivo, tenemos muchas probabilidades de tener un buen resultado en modelos de regresión logística y lineal

Imbalanced Datasets

Class Imbalance



- Number of samples per class is **not equally distributed**.
- ML model may not work well for the infrequent classes.

Examples:

- Fraud Detection
- Anomaly Detection
- Medical Diagnosis

Amazon review dataset: The number of 5 star reviews almost equals the total of the other 4 types of star reviews combined.

Class Imbalance

How to **address class imbalance** problems?

1. Down-sampling	2. Up-sampling	3. Data generation...	4. Sample weights
Reduce the size of the dominant or frequent class(es).	Increase the size of the rare or small class(es).	Create new records, <u>similar</u> but not identical.	For a model that uses a cost function, assign higher weights to rare class(es) and lower weights to dominant class(es).
	<i>random duplication</i>	<i>noise</i>	<i>Evaluation</i>

Missing Data

Handling Missing Data

Drop rows and/or columns with missing values: `df.drop(), df.dropna()`

- Less training data samples and/or less features can lead to overfitting/underfitting

Impute (fill-in) the missing values:

numeric **Average imputation** - `df['col'].fillna(df['col'].mean())` *median*

categorical **Common point imputation** - `df['col'].fillna(df['col'].mode())`

- **Placeholder** - `df['col'].fillna('missing')`

- **Advanced imputation**: Predict missing values from complete samples using ML techniques. For example, **AWS Datawig** uses neural networks to predict tabular data missing values <https://github.com/awsml/datawig>

1st detect missing data
2nd create strategy (drop,impute)

SimpleImputer in sklearn

SimpleImputer: sklearn imputation transformer for completing missing values - `.fit(), .transform()`

`SimpleImputer(missing_values=nan, strategy='mean', fill_value=None)`

- **numerical data**:
 - Strategy = "mean", replace missing values with the mean along each column
 - Strategy = "median", replace missing values with the median along each column
- **numerical or categorical data**:
 - Strategy = "most_frequent", replace missing values with the most frequent value along each column
 - Strategy = "constant", replace missing values with specified `fill_value`

Data Processing Hands-on

- Let's analyze an Amazonian dataset.
- Business Motivation:** In some marketplaces, ASINs having electrical plugs need to be accompanied by a compliance form. *Binary classification*
- ML Problem:** Predict whether an ASIN has electrical plug or not.



Amazon SageMaker

MLA-TAB-DAY1-DATA-PREPROCESSING-NB.ipynb

Feature	Description	Feature Type	Data Type
ASIN	Product ASIN	Text	string
target_label	Binary field with values in {0,1}. A value of 1 shows ASIN has a plug, otherwise 0	Numerical	integer
ASIN_STATIC_ITEM_NAME	Title of the ASIN	Text	string
ASIN_STATIC_PRODUCT_DESCRIPTION	Description of the ASIN	Text	string
ASIN_STATIC_ITEM_CLASSIFICATION	Item classification of whether it is a standalone or bundle parent item etc.	Categorical	string
ASIN_STATIC_GL_PRODUCT_GROUP_TYPE	GL product group information for the ASIN	Categorical	string
ASIN_STATIC_ITEM_PACKAGE_WEIGHT	Weight of the ASIN	Numerical	float
ASIN_STATIC_LIST_PRICE	Price information for the ASIN	Numerical	float
ASIN_STATIC_BATTERIES_INCLUDED	Information whether batteries are included along with the product	Binary	bool
ASIN_STATIC_BATTERIES_REQUIRED	Information whether batteries are required for using the product	Binary	bool

K Nearest Neighbors (KNN)

K Nearest Neighbors (KNN)

K Nearest Neighbors (KNN) predicts new data points based on K similar records from a dataset.

What class does x_2 belong to?
Look at the K closest data points:

- Choose $K = 3$



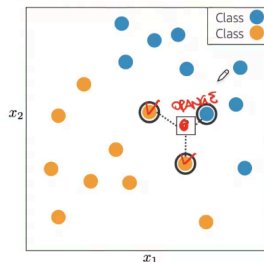
KNN is a simple algorithm used normally for imputation (missing values), not really for predictions

K Nearest Neighbors (KNN)

K Nearest Neighbors (KNN) predicts new data points based on K similar records from a dataset.

What class does x_2 belong to?
Look at the K closest data points:

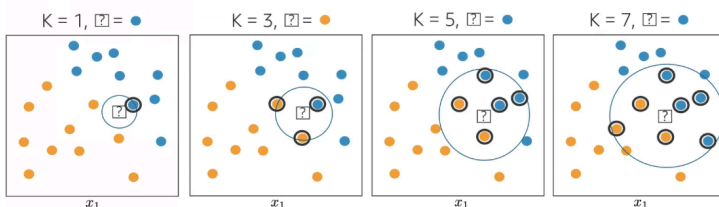
- Choose $K = 3$
- Calculate the distances from x_2 to all data points
- Find the K nearest neighbors
- Pick the majority class: ●



KNN: Choosing K *hyperparameter*

How to choose K?

- Use a validation set to select an appropriate value for K.



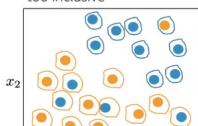
KNN: Choosing K

What is the effect of K on the predictions?

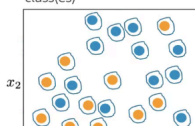
Low $K (K=1)$: **Overfitting**, predictions are greatly impacted by noise in the data (outliers, mislabeling)

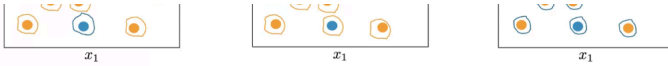


Rule of thumb ($K=\sqrt{n}$): Robust to noise, but "neighborhoods" can get too inclusive



Large $K (K=n)$: **Underfitting**, a rare class will always be outvoted by the other class(es)

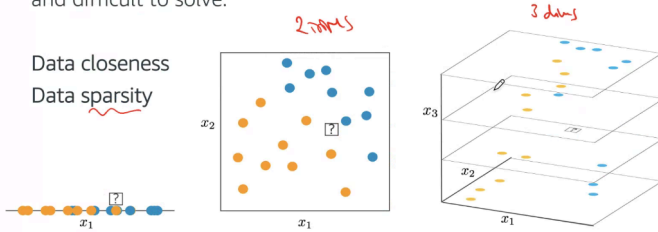




KNN: Curse of Dimensionality

With **too many features**, KNN becomes computationally expensive and difficult to solve.

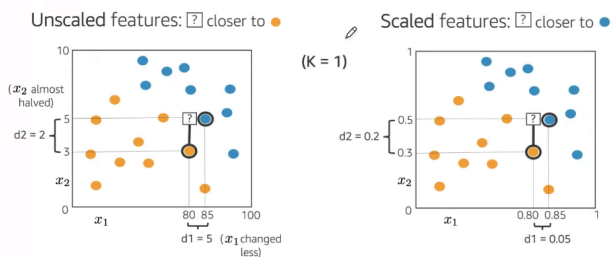
Data closeness
Data sparsity



KNN: Feature Scaling

Features should be **on the same scale** when using KNN.

Especialmente en algoritmos basados en distancia, es importante que todas las features estén en la misma escala.--> FEATURE SCALING



Feature Scaling

Motivation: Many algorithms are sensitive to features being on different scales, like metric-based algorithms (KNN, K Means) and gradient descent-based algorithms (regression, neural networks)

Note: tree-based algorithms (decision trees, random forests) do not have this issue

Solution: Bring features to the **same scale**

Common choices (both linear):

- Mean/variance standardization *Mean = 0, 5*
- MinMax scaling

Standardization in sklearn

StandardScaler: sklearn scaler, scaling values to be centered around mean 0 with standard deviation 1 - `.fit(), .transform()`

$$\text{Transform: } x_{\text{scaled}} = \frac{x - x_{\text{mean}}}{x_{\text{std}}}$$

```
from sklearn.preprocessing import StandardScaler
std_sc = StandardScaler()

raw_data = np.array([[ -3.4], [4.5], [50], [24], [3.4], [1.6]])
scaled_data = std_sc.fit_transform(raw_data)
print(scaled_data.reshape(1,-1))
```

```
[[ -0.90560498 -0.47848383  1.98151777  0.57580257 -0.53795639 -0.63527514]]
```

MinMax Scaling in sklearn

MinMaxScaler: sklearn scaler, scaling values so that minimum value is 0 and maximum value is 1 - `.fit(), .transform()`

$$\text{Transform: } x_{\text{scaled}} = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

```
from sklearn.preprocessing import MinMaxScaler
minmax_sc = MinMaxScaler()

raw_data = np.array([[ -3.4], [4.5], [50], [24], [3.4], [1.6]])
scaled_data = minmax_sc.fit_transform(raw_data)
print(scaled_data.reshape(1,-1))
```

```
[[0. 0.14794007 1. 0.51310861 0.12734082 0.09363296]]
```

KNN: Summary

- **Simple to understand** and easy to implement
- **Non-parametric**, no parameters are actually being learned
 - Useful when data may be unusual distributed
 - Successful handling classification problems where each class has many possible prototypes (decision boundary is usually quite irregular)
 - With unbalanced datasets and large K, the dominant class prevails
- **Lazy algorithm**
 - No training really, all magic happens at prediction time
 - **Computationally expensive**, not production friendly
 - Works best **less features** (dimensionality reduction) and **scaled data**

Pipeline (sklearn)

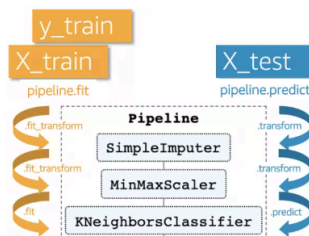
Pipeline in sklearn

Pipeline: sklearn sequential data transforms with a final estimator (prevents data leakage) - `.fit()`, `.predict()`

Pipeline(steps, verbose=False)

```
pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', MinMaxScaler()),
    ('clf', KNeighborsClassifier(n_neighbors = 3))
])
```

```
pipeline.fit(X_train, y_train)
predictions = pipeline.predict(X_test)
```



Final Project

Binary Classification

Business Problem:

Predicting substitutes is of critical importance to Amazon, for several reasons:

- When products go out of stock, we want to provide a suitable replacement
- When products are overpriced, we want to provide a reasonable alternative

Substitutable products are "similar," so a substitute model can enable other ML models that require product-to-product similarity.

Final Project

ML Problem - Product substitute: Given a pair of products (key_product, cand_product), predict whether the pair is a pair of substitute products or not

- We say that cand_product is a "substitute" for key_product if a customer would buy cand_product in place of key_product -- say, if key_product were out of stock.
- The goal of this project is to predict a substitute relationship between products in the pairs.
- Link: <https://leaderboard.corp.amazon.com/tasks/542>

Final Project Datasets

1. training.csv: training data with product pair IDs, their labels ("1" substitute, "0" not substitute), along with the pair products features

(115 features of the *key_product* of the pair, and 115 features of the *cand_product* of the pair).

	ID	label	key_Region Id	key_MarketPlace Id	key_ASIN	key_Binding Code	key_binding_description	key_brand_code	key_case_pack_quantity
0	34016	0	1	1	B00YC28IKA	kitchen	Kitchen	NICLW	NaN
1	3581	0	1	1	B00U25WT7A	office_product	Office Product	NaN	NaN
2	36025	1	1	1	B011BZ3GXU	consumer_electronics	Electronics	NaN	NaN
3	42061	1	1	1	B0089XD3J1	pc	Personal Computers	NaN	NaN
4	14628	1	1	1	B014UTSBZW	miscellaneous	Misc.	ZUKC7	1.0

5 rows x 228 columns

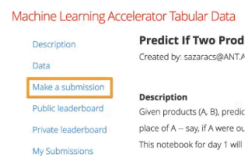
How to submit to Leaderboard?

How do I submit to Leaderboard?

- Your submission should be a csv file with the first row: "ID, label". The IDs will be the IDs from the *public_test_features.csv* file.

ID	label
35057	0
41573	0
44029	0
6462	0
17533	0

- You submit your model output (as a .csv file) to Leaderboard via the 'Make a Submission' tab.



Public and Private Leaderboard

Your submission automatically goes to two leaderboards: Public and Private.

- The Public Leaderboard is a subset of the test dataset that gives you live feedback on the performance of your model on unseen data. We keep the highest score of all your submissions. *→ Only one visible until the submission deadline*
- The Private Leaderboard is another subset of your test dataset. These results are hidden until the end of the competition. This leaderboard will be used to determine the winners of the competition. *→ Accuracy*

ML3

Final Project Walkthrough Lecture 1

- See the provided project below to get started with this project.
- Follow the given hints and directions.
- Complete the notebook and submit your results to

<https://leaderboard.corp.amazon.com/tasks/542>

PROJECT : Training dataset *→ training validation*
Test dataset *→ split already provided*
no labels *→ submit to leaderboard*



Amazon SageMaker

MLA-TAB-DAY1-FINAL-STUDENT-NB.ipynb